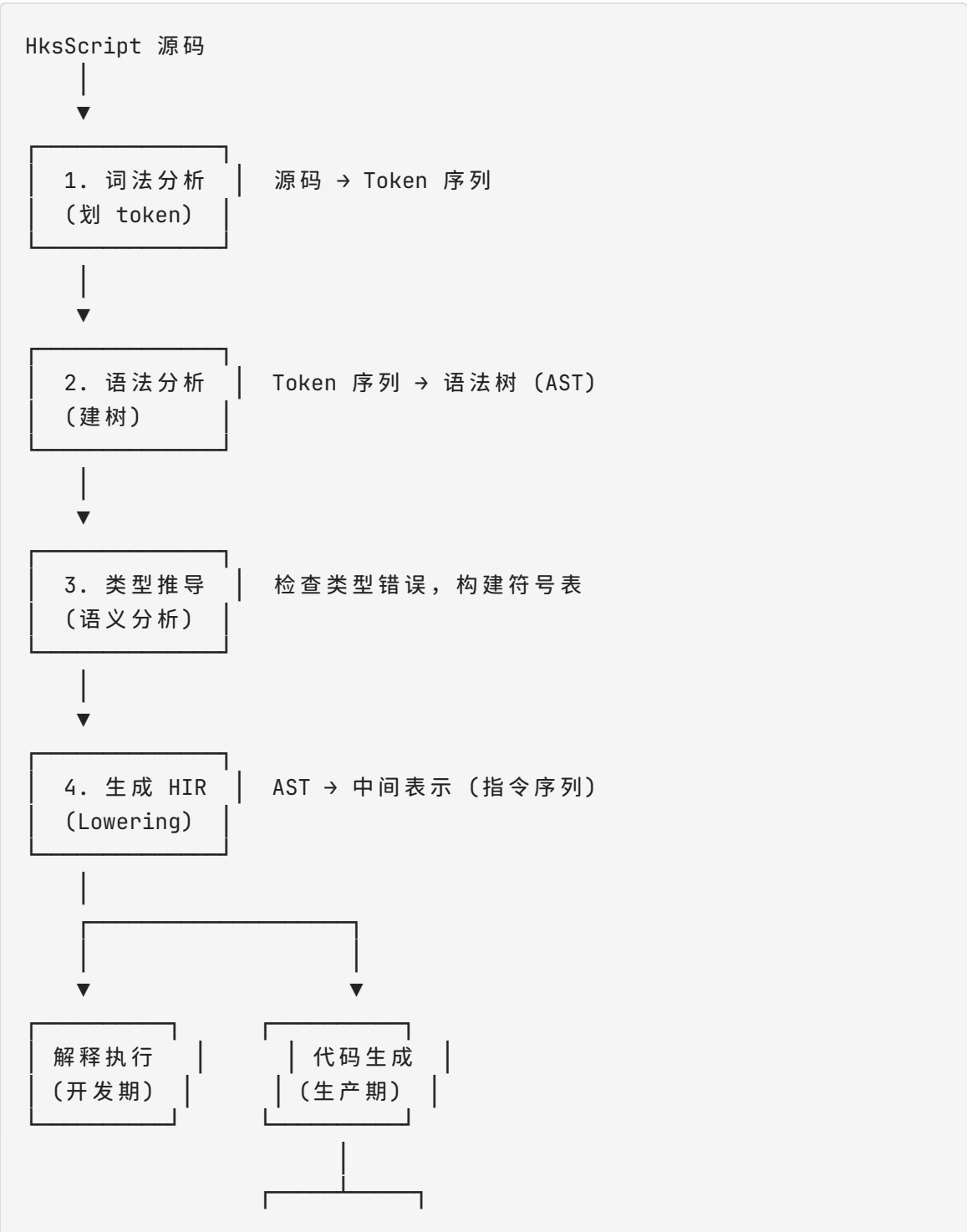


HKS脚本编译实现

整体流程



▼

C++

C#

快速导航

章节	内容
1. 词法分析	Token 类型表、ANTLR .g4 文件、TokenStreamFilter
2. 语法分析	AST 节点定义、ParseTree→AST 转换
3. 类型推导	推导规则、符号表、错误报告
4. 生成 HIR	HIR 指令定义、Lowering 规则表、Pack\<T>
5. 解释执行	HIR 解释器、Return 控制流
6. 代码生成	C++ 后端、C# 后端
7. 优化	常量折叠、死代码删除、变量内联

1. 词法分析（划 token）

把源码字符串切成一个个最小有意义单元，叫 token。

输入

```
img = load("test.png")
```

输出

```
ID("img")    → 变量名
OP("=")      → 赋值
ID("load")   → 函数名
```

```
OP("(")    → 左括号
STR("test.png") → 字符串
OP(")")    → 右括号
```

每个 token 还附带它在源码中的**位置**（行号、列号），这样报错时能指出"第 3 行第 5 列有错误"。

换行处理（语句跨行）

HksScript 用换行分隔语句（不像 C 语言用分号），但长语句需要能跨多行。规则如下：

规则一：括号不匹配时自动换行

```
# 左括号未闭合，继续读下一行
result = query from circles with (
    area > 10 and radius < 50 or
    area < 5
)

# 多层圆括号嵌套也没问题
result = f(a, g(b,
    h(c)))
```

词法分析器维护一个括号栈（当前只需要 `()` 圆括号），遇到 `(` 入栈，遇到 `)` 出栈。换行时栈为空则语句结束，栈非空则继续读下一行。

规则二：行末是运算符时自动换行

```
# 行末是 + - * / | & and or not 等运算符或逻辑关键字时，表达式未结束
total = a + b + c -
    d + e

# 逻辑条件跨行
result = query from circles with area > 10 and
    radius < 50 or
    not (x < 5)

# 管道运算符行首续行
load("batch/*.png") ⇒ find_circles() ⇒ query(area > 10)
```

实现方式：词法分析器看到换行符时，检查前一个 token 的类型：- 如果前一个 token 是运算符或逻辑关键字（+ - * / | & ⇒ and or not , ([{ }）→ 跳过换行 - 如果前一个 token 是普通 token（ID、字面量、））→ 换行即语句结束

规则三：行首以运算符开头也续行

```
result = big_query_result_arg1
      + arg2 + arg3
```

实现方式：如果换行后第一个非空 token 是运算符，跳过换行，把运算符接到上一行末尾。

三种规则的优先级（按顺序检查）：

遇到换行符 →

1. 括号计数器 > 0?

→ 跳过换行

2. 前一个 token 是运算符?

→ 跳过换行

3. 下一行的第一个有效 token 是运算符?

→ 跳过换行

4. 否则

→ 换行 = 语句结束

Token 定义

词法分析器输出以下 token 供语法分析使用：

Token	来源	说明
ID	letter+	变量名、函数名
INT	digit+	整数
FLOAT	digit+.digit+	浮点数
STRING	"..."	字符串
OP	+ - * / = \ & > < ≥ ≤ = ≠	运算符

Token	来源	说明
PIPE	⇒	管道运算符
LPAREN	(	左括号
RPAREN	)	右括号
COMMA	,	逗号
COLON	:	冒号
NEWLINE	\n	行尾，语句分隔
INDENT	缩进增加	块开始
DEDENT	缩进减少	块结束
KEYWORD	if elif else def import query from with and or not	关键字

ANTLR .g4 文件

在 .g4 中定义词法和语法规则，ANTLR 自动生成 `Lexer.cs` 和 `Parser.cs`：

```
grammar HksScript;

// 词法规则
NEWLINE: '\r'?''\n';
WS: [ \t]+ → skip;
COMMENT: '#' ~[\r\n]* → skip;
INDENT: [ \t]+; // 不由 ANTLR 处理，由自定义过滤器接管

INT: DIGIT+;
FLOAT: DIGIT+ '.' DIGIT+;
STRING: '"' (~["\\] | '\\\'' .)* '"';
ID: LETTER (LETTER | DIGIT)*;

OP: '+' | '-' | '*' | '/' | '|' | '&' | '=' | '>' | '<' | '≥' | '≤' | '==' | '≠';
```

```

PIPE: '⇒';
LPAREN: '('; RPAREN: ')'; COMMA: ','; COLON: ':';

KW_IF: 'if'; KW_ELIF: 'elif'; KW_ELSE: 'else';
KW_DEF: 'def'; KW_IMPORT: 'import'; KW_QUERY: 'query';
KW_FROM: 'from'; KW_WITH: 'with'; KW_AND: 'and'; KW_OR: 'or'; KW_NOT: 'not';

fragment DIGIT: [0-9];
fragment LETTER: [a-zA-Z_];

// 语法规则（使用 INDENT/DEDENT）
program: statement* EOF;

statement
    : NEWLINE                                // 空行
    | importStmt NEWLINE
    | assignStmt NEWLINE
    | exprStmt NEWLINE
    | ifStmt
    | funcDef
    ;

block: INDENT statement+ DEDENT;

importStmt: KW_IMPORT ID (',' ID)*;
assignStmt: ID '=' expr;
exprStmt: expr;

ifStmt: KW_IF expr COLON NEWLINE block
       (KW_ELIF expr COLON NEWLINE block)*
       (KW_ELSE COLON NEWLINE block)?;

funcDef: KW_DEF ID '(' ')' COLON NEWLINE block;

expr: ...

```

注意点： - `INDENT` 和 `DEDENT` 不由 ANTLR 词法分析器产生，而是由自定义的 `TokenStreamFilter` 插入 - `NEWLINE` 需要在续行规则中被删除 - `WS` 和 `COMMENT` 直接跳过，缩进信息由 `INDENT` 和 `DEDENT` 携带

TokenStreamFilter（词法分析器和语法分析器之间的过滤器）

ANTLR 的词法分析器是上下文无关的，无法原生处理 Python 风格的 INDENT/DEDENT。需要在词法分析器和语法分析器之间加一个自定义过滤器：

ANTLR Lexer → 原始 Token 流 → TokenStreamFilter → 干净 Token 流 → ANTLR Parser

过滤器的工作流程：

```
function Filter(originalTokens):
    stack = [0]          // 缩进栈，初始为 0
    result = []
    i = 0

    while i < len(originalTokens):
        tok = originalTokens[i]

        // 1. 跳过注释和空格（已在 ANTLR 中 skip，此处仅做安全处理）
        if tok is COMMENT or tok is WS: i++; continue

        // 2. 续行判断
        if tok is NEWLINE:
            // 检查括号栈
            if bracketStack > 0:
                i++; continue
            // 检查前一个 token
            if prevToken is OP or PIPE or COMMA or COLON or AND or OR:
                i++; continue
            // 检查后一个 token
            if nextNonWhitespace is OP or PIPE or AND or OR:
                i++; continue

            // 有效 NEWLINE - 处理缩进
            indentLevel = countIndent(nextLine)
            if indentLevel > stack[-1]:
                result.push(INDENT)
                stack.push(indentLevel)
            elif indentLevel < stack[-1]:
                while indentLevel < stack[-1]:
                    result.push(DEDENT)
                    stack.pop()
            result.push(NEWLINE)
            i++; continue
```

```

        // 3. 普通 token，直接输出
        result.push(tok)
        prevToken = tok
        i++

    // 文件结束，弹出剩余缩进
    while len(stack) > 1:
        result.push(DEDEDENT)
        stack.pop()

    return result

```

这个过滤器实现三个功能：1. **续行处理** — 括号栈、行末/行首运算符检查，决定是否跳过 NEWLINE 2. **缩进处理** — 遇到有效 NEWLINE 时，计算行缩进并与栈顶比较，插入 INDENT/DEDEDENT 3. **块结束** — 文件末尾自动补齐 DEDEDENT 确保所有块闭合

ANTLR 中可以通过继承 `TokenStreamRewriter` 或自定义 `ITokenStream` 接口来实现这个过滤器。

把第一步得到的 token 序列，按照语法规则搭成**树状结构**，叫 AST（Abstract Syntax Tree，抽象语法树）。

为什么是树不是序列

因为代码有嵌套结构：

```
a = 1 + 2 * 3
```

如果线性处理 token，不知道先加还是先乘。搭成树之后，树的结构天然表达了优先级：

```

      =
     /\
    a  +
      /\
     1  *
      /\
     2  3

```


遍历时先走子树（`2 * 3`），再算父节点（`+`），优先级自然正确。

对于 query 语法糖

```
res = query from circles with area > 10
```

搭成 AST 后，`query from ... with ...` 这种特殊写法会变成一个 `QueryExpr` 节点，和函数调用 `query(circles, area > 10)` 是不同的 AST 节点。到第 4 步 Lowering 才会把它们统一。

AST 节点定义

ANTLR 生成的 ParseTree 是具体的语法树，包含了很多括号、分号等冗余信息。我们会在语法分析之后把 ParseTree 转成更干净的 AST。以下是 AST 的节点定义（C# record）：

```
// 程序 = 一组语句
record Program(List<Stmt> Statements);

// 语句基类
abstract record Stmt();

// 赋值语句：a = 1 + 2
record Assign(string Name, Expr Value) : Stmt();

// 函数定义：def fn(a, b) → Set<Circle>: body
// ReturnType 为 null 表示无返回值
record FuncDef(string Name, List<string> Params, Type? ReturnType, List<Stmt> Body) : Stmt();

// 导入语句：import find_circle
record Import(string Name) : Stmt();
record Import(List<string> Names) : Stmt(); // import a, b, c 的批量写法

// If 语句
record If(Expr Condition, List<Stmt> Then, List<ElifClause> Elifs, List<Stmt>? Else) : Stmt();

record ElifClause(Expr Condition, List<Stmt> Body);

// Return 语句：return x
// 函数没有 return 则无返回值，等同于 return void
record Return(Expr? Value) : Stmt();
```

```

// 表达式基类
abstract record Expr();

// 字面量: 1, 3.14, "hello", true
record Literal(object Value) : Expr();

// 变量引用: img, circles, area
record Variable(string Name) : Expr();

// 二元运算: a + b, area > 10, 1 + 2 * 3
record Binary(BinaryOp Op, Expr Left, Expr Right) : Expr();

enum BinaryOp { Add, Sub, Mul, Div, Lt, Gt, Eq, Neq, Le, Ge, And, Or }

// 一元运算: -x, not condition
record Unary(UnaryOp Op, Expr Operand) : Expr();

enum UnaryOp { Neg, Not }

// 函数调用: load("test.png"), find_circles(img)
record Call(string Name, List<Expr> Args) : Expr();

// query 语法糖: query from circles with area > 10
// 等价于 Call("query", [circles, area > 10]), 但 AST 保留原始写法
record QueryFrom(Expr Collection, Expr Condition) : Expr();

// 管道: a ⇒ f(b) → 上一步结果自动填入 f 的第一个类型兼容参数
// 等价于 f(pipe_result, b), 即使 f 只有一个参数也要写 f()
record Pipe(Expr Left, Expr Right) : Expr();

// --- 集合运算 (运算符语法糖) ---
// a | b → Union(a, b), a & b → Intersect(a, b), a - b → Diff(a, b)
// 三者与数值运算共享词法符号, 类型推导时根据操作数类型区分

// --- 集合相关 ---

// 范围: range(col)
record Range(Expr? Start, Expr? End) : Expr(); // 省略 start 则从 0 开始

```

在项目里怎么做

同样用 ANTLR, 在 `.g4` 文件里定义语法规则:

```
expr
: expr '⇒' expr      # pipeExpr
| expr '+' expr      # addExpr
| ID '(' exprList? ')' # callExpr
...
```

ANTLR 自动生成 `Parser.cs` 和 `Visitor` 接口。你写的 `Evaluator.cs` 就是继承 `Visitor`，遍历 AST 节点做处理。

当前 `Evaluator` 的处理方式：

```
// 访问加法节点
VisitAddExpr(ctx) {
    left = Visit(ctx.expr(0)); // 先算左边
    right = Visit(ctx.expr(1)); // 再算右边
    return left + right;       // 返回结果
}
```

3. 类型推导

AST 搭建完了，但还不知道每个节点的**类型**。类型推导就是遍历 AST 给每个表达式算出类型，同时检查有没有类型错误。

推导方式

从已知的东西往未知推：

已知：

- 字面量 "test.png" 类型是 string
- load 函数接受 string，返回 Mat
- area 是特征函数，类型是 Circle → float

推导：

load("test.png")	→ 返回 Mat	所以 img 是 Mat
find_circles(img)	→ 返回 Set<Circle>	所以 circles 是 Set<Circle>
area > 10	→ area(Circle) > 10	所以结果是 bool
	query (筛选条件)	

报错

```
a = 1 + "hello"    # 错误：整数 + 字符串，不匹配
b = unknown       # 错误：未定义的变量
```

符号表

类型推导过程中会维护一个**符号表**，记录所有变量名及其类型：

变量名	类型
img	Mat
circles	List<Circle>
area	Circle → float
res	List<Circle>

后续步骤（Lowering、代码生成）都会查这个符号表。

4. 生成 HIR（中间表示）

AST 包含了太多**语法细节**——比如 `query from ... with ...` 和 `query(...)` 在 AST 里是两种节点，但它们做的事情完全一样。代码生成时不想关心这些区别，所以需要 HIR（High-level Intermediate Representation，高层次中间表示）。

HIR 是什么

HIR 是一个**扁平的指令序列**，每条指令做一件事，不嵌套。

AST（树状，有嵌套）：

```
=
/ \
```

```
img load
  |
  "test.png"
```

HIR（扁平，每条指令有编号）：

```
t0 = Load("test.png")
t1 = Let("img", t0)
```

为什么用 HIR

对比	AST	HIR
结构	树，有嵌套	列表，扁平
语法糖	保留原始写法	已经被统一
类型	可能未推导	每条指令都带类型
代码生成	要递归遍历，麻烦	顺序遍历，直接对映到目标代码

以 query 语法的两种写法为例

写法一：

```
r = query from circles with area > 10
```

写法二：

```
r = query(circles, area > 10)
```

在 AST 里，写法一是 `QueryFromStmt` 节点，写法二是 `CallExpr` 节点，完全不同。

但 Lowering 之后，两者变成**相同的 HIR**：

```
t0 = Load("test.png")
t1 = Call(find_circles, t0)
t2 = Let("circles", t1)
t3 = Filter(t2, area > 10)    ← 两种写法的统一产物
t4 = Let("r", t3)
```

代码生成器只需要认识 `Filter` 指令，不需要关心用户当初用的是语法糖还是函数调用。

HIR 指令定义

每条 HIR 指令都有一个唯一的 `Id` 编号，表示这条指令产生的结果。后续指令可以通过编号引用前面的结果。

HIR 只保留最核心的控制流和运算，所有 IO、集合操作、打印等一律用函数调用表示。

```
// 程序 = 指令序列
record ProgramHIR(List<HIRInst> Instructions);

// 指令基类
abstract record HIRInst
{
    Id Dest;  // 结果编号，如 t0, t1, t2...
}

typealias Id = int;  // t0 = 0, t1 = 1 ...

// 常量: t0 = Const(42, int)
record Const(Id Dest, Type Ty, object Value) : HIRInst();

// 二元运算: t2 = Add(t0, t1)
// 包括算术 (Add/Sub/Mul/Div) 和比较 (Lt/Gt/Eq...)
record Binary(Id Dest, Type Ty, BinaryOp Op, Id Left, Id Right) : HIRInst();

enum BinaryOp { Add, Sub, Mul, Div, Lt, Gt, Eq, Neq, Le, Ge, And, Or }

// 一元运算: t1 = Neg(t0)
record Unary(Id Dest, Type Ty, UnaryOp Op, Id Operand) : HIRInst();

enum UnaryOp { Neg, Not }
```

```

// 函数调用: t3 = Call("find_circles", [t1])
// 所有 IO (load/save/print/query/save)、集合运算 (union/intersect/diff)
// 都走 Call, 不设专用指令
record Call(Id Dest, Type RetTy, string Name, List<Id> Args) : HIRInst();

// 变量绑定: t2 = Let("x", t0)
// 把值 t0 绑定到变量名 "x"
record Let(Id Dest, string Name, Id Source) : HIRInst();

// 分支: if (t0) { thenBlock } else { elseBlock }
// Block 是 HIR 指令列表的子集
record Branch(Id Dest, Expr Condition, Block ThenBlock, Block? ElseBlock) : HIRInst();

// 函数返回: Return(t0)
// 函数立即结束并返回 t0 的值, 无参 Return() 表示 void 返回
record Return(Id? Value) : HIRInst();

// 模块导入: Import("find_circle")
// 加载并初始化模块, 注册模块中的函数到符号表
record Import(Id Dest, string Name) : HIRInst();

```

每条指令的 `Id` 在整个程序里递增。变量绑定 `Let` 的 `Dest` 是它自己的 `Id`, `Name` 是变量名, 之后可以用变量名引用这个值。例如:

```

t0 = Load("test.png")           // Dest=0
t1 = Let("img", t0)              // "img" → t0, Dest=1
t2 = Call("find_circles", t1)    // 参数 t1 → 实际用 t0 的值

```

这里的 `Let("img", t0)` 意思是给 `t0` 的结果起个别名叫 `"img"`。之后代码里提到 `img` 就会查到实际指向 `t0`。

Lowering 规则 (AST → HIR 的映射表)

AST 节点	生成的 HIR
<code>import a</code>	<code>t0 = Import("a")</code> — 加载并初始化模块
<code>import a, b</code>	<code>t0 = Import("a") t1 = Import("b")</code>
<code>a = 1</code>	<code>t0 = Const(1, int) t1 = Let("a", t0)</code>

AST 节点	生成的 HIR
<code>a = b + c</code>	<code>t0 = Binary(Add, b, c)</code> <code>t1 = Let("a", t0)</code>
<code>a + b</code>	<code>t0 = Binary(Add, a, b)</code>
<code>-x</code>	<code>t0 = Unary(Neg, x)</code>
<code>not c</code>	<code>t0 = Unary(Not, c)</code>
<code>fn(a, b)</code>	<code>t0 = Call(fn, [a, b])</code>
<code>a ⇒ f()</code>	<code>t0 = Call(f, [a])</code> — pipe 结果填入第一个参数
<code>if c: ...</code> <code>else: ...</code>	<code>t0 =</code> <code>Branch(c, ...block1..., ...block2...)</code>
<code>return x</code>	<code>t0 = Return(x)</code>
<code>return</code>	<code>t0 = Return()</code>

所有 IO 操作、集合操作、query、import 等一律走 `Call`：

语法	生成的 HIR
<code>load("x.png")</code>	<code>t0 = Call(load, ["x.png"])</code>
<code>save(img, p)</code>	<code>t0 = Call(save, [img, p])</code>
<code>print(x)</code>	<code>t0 = Call(print, [x])</code>
<code>query from x with c</code>	<code>t0 = Call(query, [x, c])</code>
<code>query(x, c)</code>	<code>t0 = Call(query, [x, c])</code>
<code>range(x)</code>	<code>t0 = Call(range, [x])</code>
<code>union(a, b) / a \ b</code>	<code>t0 = Call(union, [a, b])</code>
<code>intersect(a, b) / a & b</code>	<code>t0 = Call(intersect, [a, b])</code>

语法	生成的 HIR
<code>diff(a, b) / a - b</code>	<code>t0 = Call(diff, [a, b])</code>
<code>sql(conn, stmt)</code>	<code>t0 = Call(sql, [conn, stmt])</code>

每条 HIR 指令的记录格式：

```
record HIRInst
{
    Id Dest;           // 结果编号 (t0, t1, t2...)
    OpKind Op;         // 操作类型 (Load, Call, Filter, Add...)
    Type ResultType;   // 结果类型 (Mat, Set<Circle>, int...)
    object[] Args;     // 参数列表 (可以是 Id 引用, 也可以是常量)
}
```

Pack 统一容器

所有运行时数据用 `Pack<T>` 封装，消除单值/多值的类型系统差异：

```
abstract record Pack<T>;
record One<T>(T Value)      : Pack<T>; // 单值
record Many<T>(List<T> Items) : Pack<T>; // 多值
```

函数签名只写一套：

```
find_circles : Pack<Mat> → Pack<Circle>
query       : Pack<T>, Expr → Pack<T>
```

函数内部检查 `Pack` 类型： - `One<T>` → 直接处理单值 - `Many<T>` → 遍历每个元素处理，可自动并行

执行引擎对 `Many<T>` 且算子无副作用时，自动分片并行：

```
load("batch/*.png")    → Many<Mat>
⇒ find_circles()       → 4 线程并行执行
⇒ query(area > 10)     → 并行筛选
```

并行策略由执行器/代码生成器自行决定，不作为 HIR 语义的一部分（后续可加注解机制）。

5. 解释执行

HIR 不需要编译成机器码，可以直接用一个循环逐条执行。

怎么做

```
Dictionary<Id, object?> values = new();

foreach (var inst in program)
{
    switch (inst.Op)
    {
        case OpKind.Load:
            values[inst.Dest] = Mat.FromFile((string)inst.Args[0]);
            break;

        case OpKind.Call:
            var func = LookupFunction((string)inst.Args[0]);
            var args = inst.Args.Skip(1).Select(a => values[(Id)a]);
            values[inst.Dest] = func(args);
            break;

        case OpKind.Filter:
            var list = values[(Id)inst.Args[0]] as Set<Circle>;
            var pred = BuildPredicate(inst.Condition);
            values[inst.Dest] = list.Where(c => pred(c)).ToList();
            break;

        case OpKind.Let:
            var varName = (string)inst.Args[0];
            var val = values[(Id)inst.Args[1]];
            env[varName] = val;
            values[inst.Dest] = val;
            break;
    }
}
```

Return 的处理

函数调用遇到 `Return` 时立即停止执行。在解释器中用一个标记实现控制流跳出：

```
// 全局返回标记，遇到 return 时设置
private bool _returned = false;
private object? _returnValue = null;

void ExecuteFunction(FuncDef func, object?[] args)
{
    env.EnterScope();
    foreach (var (param, arg) in func.Params.Zip(args))
        env[param] = arg;

    _returned = false;
    _returnValue = null;

    foreach (var inst in func.Body)
    {
        Execute(inst);
        if (_returned) break; // 遇到 return 跳出循环
    }

    env.ExitScope();
    return _returnValue; // 没有 return 则返回 null
}

// Return 指令的执行：
case OpKind.Return:
    _returned = true;
    _returnValue = inst.Value != null ? values[inst.Value] : null;
    break;
```

在 HIR 解释器的循环中，`Return` 不做值传递，只设置标记。上层函数执行器检查标记后停止执行。

这和当前 `Evaluator.cs` 做的事情本质上一样，只是现在的 `Evaluator` 是递归走 AST，改成走 HIR 列表更简单。

6. 代码生成 (HIR → C++ / C#)

基本思路

遍历 HIR 指令列表，每条指令对应生成几行目标语言代码。

HIR → C++ 示例

```
# HksScript 源码
img = load("test.png")
circles = find_circles(img)
r = query from circles with area > 10
```

```
// HIR
t0 = Load("test.png")
t1 = Let("img", t0)
t2 = Call("find_circles", t1)
t3 = Let("circles", t2)
t4 = Filter(t3, area > 10)
t5 = Let("r", t4)
```

```
// 生成的 C++
auto t0 = cv::imread("test.png");
auto img = t0;
auto t2 = find_circles(t0); // 传给 C++ 算法库
auto circles = t2;
auto t4 = filter(circles, [](const Circle& c) { return c.area > 10; });
auto r = t4;
```

HIR → C# 示例

```
// 生成的 C#
var t0 = Mat.FromFile("test.png");
var img = t0;
var t2 = Find.FindCircle(t0); // P/Invoke 调 C++ 算法库
var circles = t2;
```

```
var t4 = circles.Where(c => c.Area > 10).ToList();
var r = t4;
```

代码生成器的结构

```
class CodeGenCpp
{
    string Generate(Program program)
    {
        StringBuilder sb = new();
        int nextId = 0;

        foreach (var inst in program.Instructions)
        {
            sb.AppendLine(inst switch
            {
                Load(var path) =>
                    $" auto t{nextId++} = cv::imread(\"{path}\");",

                Call(var func, var args) =>
                    $" auto t{nextId++} = {func}({Join(args)});",

                Filter(var source, var cond) =>
                    $" auto t{nextId++} = filter({source}, {EmitCondition(cond)});",

                Let(var name, var source) =>
                    $" auto {name} = {source};",

                _ => throw new NotSupportedException()
            });
        }

        return sb.ToString();
    }
}
```

关键点：

- **每条 HIR 指令独立生成一行或多行代码**
 - 不需要考虑上下文，因为上下文已经被 HIR 扁平化了
 - 指令之间的数据依赖通过 `t0`、`t1` 这样的临时变量名传递
-

7. 优化（TODO — 后续版本实现）

当前状态：以下优化策略是设计规划，短期内不会实现。第一期先实现完整的 HIR 生成和代码正确性，优化在后续版本逐步加入。当前解释执行阶段可直接运行 HIR，不需要优化。

HIR 是扁平指令序列，可以做简单的优化。优化**不能改变程序的计算顺序**（图像处理顺序必须保留），但可以去掉无用计算和合并操作。

常量折叠

```
# 原始 HIR
t0 = Const(10)
t1 = Const(20)
t2 = Add(t0, t1)

# 优化后（编译时直接算出 30）
t2 = Const(30)
```

死代码删除

如果一个 `tN` 从来没有被引用，删掉它：

```
# 原始
t0 = Load("a.png")    # 没用过，删掉
t1 = Load("b.png")
t2 = find_circles(t1)

# 优化后
t1 = Load("b.png")
t2 = find_circles(t1)
```

冗余变量消除

如果一个变量只是另一个变量的别名，直接用原名：

```
# 原始
t2 = Call(find_circles, t1)
t3 = Let("circles", t2)
t4 = Filter(t3, ...)

# 优化后
t2 = Call(find_circles, t1)
t4 = Filter(t2, ...)    # 直接用 t2, 跳过 "circles"
```

变量内联（目标语言相关）

内联决定代码可读性的上限。但不同目标语言的语义模型不同，内联策略必须区分：

```
// HIR
t0 = Load("test.png")
t1 = Call("find_circles", t0)
t2 = Filter(t1, area > 10)
```

C# 后端（引用语义） — 激进内联

C# 全是引用，临时对象不存在生命周期问题，多层嵌套就是多层函数调用：

```
auto circles = find_circles(Mat.FromFile("test.png"));
var r = circles.Where(c => c.Area > 10).ToList();
```

规则：- 只被引用一次的 `tN` 全部内联 - 被多个指令共享的 `tN` 保留具名变量 - 用户自定义的变量名保留不内联（`Let("img", ...)` 中的 `img`）

C++ 后端（值语义） — 保守内联

C++ 的临时对象生命周期只到表达式末尾，无脑内联会导致：

```
// 无脑内联后的 C++：
auto r = filter(
    find_circles(cv::imread("test.png")), // Mat 临时对象
    [](auto& c) { return c.area > 10; }
);
```

```
// ↑ imread 返回的 Mat 在这个分号之前就析构了
// ↑ 如果 find_circles 内部持有 Mat 的引用，就悬空了
```

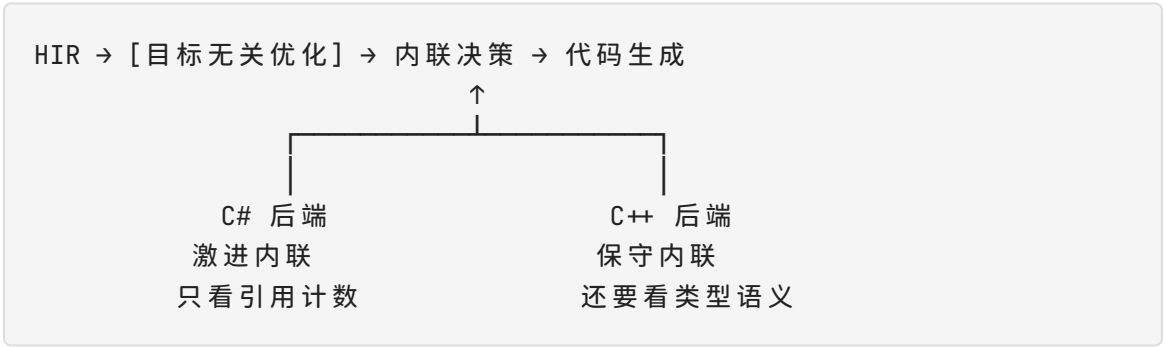
规则：

类型	内联策略	举例
POD (int/float/bool)	激进内联	<code>t0 = Const(42)</code> → 直接 <code>42</code>
对象 (Mat/Circle/List)	保留具名变量	<code>auto img = cv::imread(...)</code>
函数参数	只看一层的浅内联	<code>fn(a + b, c)</code> 可内联， <code>fn(load(path))</code> 不内联
被多次引用的	总保留	<code>auto t0 = imread(...)</code>

生成：

```
auto img = cv::imread("test.png");
auto circles = find_circles(img); // 保留具名，生命周期明确
auto r = filter(circles, [](auto& c) { return c.area > 10; });
```

两条后端路径总结



总结

源码 → Token → AST → [类型推导] → HIR → 目标代码

开发期：HIR → 逐条解释执行（当前 Evaluator 思路）

生产期：HIR → C++/C# 代码编译成 native（极致性能）

核心思路：语法写的多样性在 AST 里保留，在往 HIR 转化时统一，代码生成器只认识 HIR，不用管源语法的花样。